



北京邮电大学

Beijing University of Posts and Telecommunications

BUPT Compilers Project1 2024Fall

Name: 项 枫

Student ID: 2022211570

Class Number: 2022211801

Container Number:

6eca9c2b53e9aaa0c65b6af6613814be05cc0d8ccc63b7df064c4a7230aab09f

1 Overview

In BUPT Compilers projects, I will write compiler for a toy programming language called BPL, the abbreviation for BUPT Programming Language. BPL is a C-like programming language that removes most advanced features in C standard, such as macros or pointers. It can generate an abstract syntax tree (AST) for a given BPL program, and recover from both lexical and syntax errors. Besides, it can recognize both single-line and multi-line comments.

2 AST

2.1 AST generation

Data structures to represent the AST:

```
typedef struct Head {  
    char *type;  
    size_t line_no;  
    char *property;  
    struct Head *child, *sibling;  
} *Node, *Tree;
```

Functions to generate the AST:

```
Node new_node(const char *type, size_t line_no, const char *property, Node child, Node  
sibling);  
void print_tree(FILE *restrict stream, Tree tree, size_t indent);  
void empty_tree(Tree tree);
```

create_node is used to create a new node, where type is the type of the node, line_no is the line number of the node's location, property is the property of the node, child is the eldest child of the node, and sibling is the next sibling of the node; print_tree uses depth-first traversal to print the AST to a given stream, where stream specifies the output stream, tree is the tree to print, and indent is the number of spaces to indent; delete_tree uses a recursive method to free the memory allocated for the AST, where tree is the tree to free.

2.1 Code

The implementation of tree.h is as follows.

```
#ifndef TREE_H_  
#define TREE_H_  
  
#ifdef __cplusplus  
extern "C" {
```

```
#endif

#include <stddef.h>
#include <stdio.h>

typedef struct Head {
    char *type;
    size_t line_no;
    char *property;
    struct Head *child, *sibling;
} *Node, *Tree;

Node new_node(const char *type, size_t line_no, const char *property, Node child, Node
sibling);
void print_tree(FILE *restrict stream, Tree tree, size_t indent);
void empty_tree(Tree tree);

#ifdef __cplusplus
}
#endif

#endif
```

The implementation of tree.c is as follows.

```
#include "tree.h"
#include <stdlib.h>
#include <string.h>
#define INDENT "  "

Node new_node(const char *type, size_t line_no, const char *property, Node child, Node
sibling) {
    Node retval = calloc(1, sizeof(struct Head));
    retval->line_no = line_no;
    if (type) {
        retval->type = strdup(type);
    }
    if (property) {
        retval->property = strdup(property);
    }
    if (child) {
        retval->child = child;
    }
    if (sibling) {
        retval->sibling = sibling;
    }
}
```

```
}  
return retval;  
}  
  
void print_tree(FILE *restrict stream, Tree tree, size_t indent) {  
    if (!tree) {  
        return;  
    }  
    if (!tree->type) {  
        print_tree(stream, tree->sibling, indent);  
        return;  
    }  
    for (int i = 0; i < indent; i++) {  
        fprintf(stream, INDENT);  
    }  
    printf("%s", tree->type);  
    if (tree->child) {  
        fprintf(stream, " (%lu)", tree->line_no);  
    }  
    else if (tree->property) {  
        fprintf(stream, ": %s", tree->property);  
    }  
    putc('\n', stream);  
    print_tree(stream, tree->child, indent + 1);  
    print_tree(stream, tree->sibling, indent);  
}  
  
void empty_tree(Tree tree) {  
    if (!tree) {  
        return;  
    }  
    empty_tree(tree->child);  
    empty_tree(tree->sibling);  
    if (tree->type) {  
        free(tree->type);  
    }  
    if (tree->property) {  
        free(tree->property);  
    }  
    if (tree->child) {  
        free(tree->child);  
    }  
    if (tree->sibling) {  
        free(tree->sibling);  
    }  
}
```

```

    }
}

#ifdef TREE_C_
#include <mcheck.h>
int main(int argc, char **argv, char **env) {
    mtrace();
    Tree tree = new_node(
        "Program", 1, NULL,
        new_node(
            "ExtDefList", 1, NULL,
            new_node(
                "ExtDef", 1, NULL,
                new_node(
                    "Specifier", 1, NULL,
                    new_node(
                        "TYPE", 1, "int", NULL, NULL),
                    new_node(
                        "FunDec", 1, NULL,
                        new_node(
                            "ID", 1, "test_1_r01", NULL,
                            new_node(
                                "LP", 1, NULL, NULL,
                                new_node(
                                    "RP", 1, NULL, NULL,
                                    new_node(
                                        "SEMI", 1, NULL, NULL, NULL
                                    )
                                )
                            )
                        )
                    )
                )
            ),
            NULL),
        NULL),
    NULL),
    NULL);
    print_tree(stdout, tree, 0);
    empty_tree(tree);
    free(tree);
    muntrace();
    return 0;
}
#endif

```

3 Lexical Analysis

I used flex to generate a lexical analyzer for BPL. It can recognize lexemes and generate tokens for the parser. Tokens will be passed to the parser as nodes.

The implementation of lex.l is as follows.

```
%{
    #include <stdlib.h>
    #include "syntax.tab.h"
    #include "tree.h"
    #include "error.h"
    #define BUFLen (4096)
    size_t line = 1;
    char buffer[BUFLen];
    char *parse_int(char *raw);
    #define CONCAT(STR1, STR2) strncat(strncpy(buffer, STR1, BUFLen - 1), STR2, BUFLen - 1 - strlen(buffer))
}%
%option noyywrap
literal_int    [0-9]+|0x[0-9a-fA-F]+
literal_float  [0-9]+."[0-9]+
identifier     [a-zA-Z_][a-zA-Z0-9_]*
literal_char   \'.\'|\'\'\\x[0-9a-fA-F]{2}\\'
%%

"//".*$      { /* ignore comments */ }
"/*"([^\]|[*][^/]|[*][*]+[^/])**/"    { for (size_t i = 0; i < strlen(yytext); i++) if
(yytext[i] == '\n') line++; }
[ \t\r]+     { /* ignore whitespace */ }
"\n"         { line++; }
"int"        { yylval.node = new_node("TYPE", line, yytext, NULL, NULL); return
TYPE; }
"float"      { yylval.node = new_node("TYPE", line, yytext, NULL, NULL); return
TYPE; }
"char"       { yylval.node = new_node("TYPE", line, yytext, NULL, NULL); return
TYPE; }
"struct"     { yylval.node = new_node("STRUCT", line, NULL, NULL, NULL); return
STRUCT; }
"if"         { yylval.node = new_node("IF", line, NULL, NULL, NULL); return IF; }
"else"       { yylval.node = new_node("ELSE", line, NULL, NULL, NULL); return ELSE; }
"while"      { yylval.node = new_node("WHILE", line, NULL, NULL, NULL); return
WHILE; }
"return"     { yylval.node = new_node("RETURN", line, NULL, NULL, NULL); return
RETURN; }
"."          { yylval.node = new_node("DOT", line, NULL, NULL, NULL); return DOT; }
";"          { yylval.node = new_node("SEMI", line, NULL, NULL, NULL); return SEMI; }
```

```

", "      { yylval.node = new_node("COMMA", line, NULL, NULL, NULL); return COMMA; }
"="      { yylval.node = new_node("ASSIGN", line, NULL, NULL, NULL); return
ASSIGN; }
"<"      { yylval.node = new_node("LT", line, NULL, NULL, NULL); return LT; }
">"      { yylval.node = new_node("GT", line, NULL, NULL, NULL); return GT; }
"<="     { yylval.node = new_node("LE", line, NULL, NULL, NULL); return LE; }
">="     { yylval.node = new_node("GE", line, NULL, NULL, NULL); return GE; }
"!="     { yylval.node = new_node("NE", line, NULL, NULL, NULL); return NE; }
"=="     { yylval.node = new_node("EQ", line, NULL, NULL, NULL); return EQ; }
"+"      { yylval.node = new_node("PLUS", line, NULL, NULL, NULL); return PLUS; }
"-"      { yylval.node = new_node("MINUS", line, NULL, NULL, NULL); return MINUS; }
"*"      { yylval.node = new_node("MUL", line, NULL, NULL, NULL); return MUL; }
"/"      { yylval.node = new_node("DIV", line, NULL, NULL, NULL); return DIV; }
"&&"     { yylval.node = new_node("AND", line, NULL, NULL, NULL); return AND; }
"||"     { yylval.node = new_node("OR", line, NULL, NULL, NULL); return OR; }
"!"      { yylval.node = new_node("NOT", line, NULL, NULL, NULL); return NOT; }
"("      { yylval.node = new_node("LP", line, NULL, NULL, NULL); return LP; }
")"      { yylval.node = new_node("RP", line, NULL, NULL, NULL); return RP; }
"["      { yylval.node = new_node("LB", line, NULL, NULL, NULL); return LB; }
"]"      { yylval.node = new_node("RB", line, NULL, NULL, NULL); return RB; }
"{"      { yylval.node = new_node("LC", line, NULL, NULL, NULL); return LC; }
"}"      { yylval.node = new_node("RC", line, NULL, NULL, NULL); return RC; }
{identifier} { yylval.node = new_node("ID", line, yytext, NULL, NULL); return ID; }
{literal_char} { yylval.node = new_node("CHAR", line, yytext, NULL, NULL); return
CHAR; }
{literal_int} { yylval.node = new_node("INT", line, parse_int(yytext), NULL, NULL);
return INT; }
{literal_float} { yylval.node = new_node("FLOAT", line, yytext, NULL, NULL); return
FLOAT; }
'\\x.*'    { yylval.node = new_node("ILLEGAL", line, CONCAT("illegal char ",
yytext), NULL, NULL); return ILLEGAL; }
0x([0-9a-fA-F]|[g-zG-Z])+ { yylval.node = new_node("ILLEGAL", line, CONCAT("illegal
hex ", yytext), NULL, NULL); return ILLEGAL; }
[0-9]{identifier} { yylval.node = new_node("ILLEGAL", line, CONCAT("illegal identifier
", yytext), NULL, NULL); return ILLEGAL; }
.          { yylval.node = new_node("ILLEGAL", line, CONCAT("illegal symbol ",
yytext), NULL, NULL); return ILLEGAL; }
%%

char *parse_int(char *raw) {
    if (raw[1] == 'x') {
        int val = 0;
        char *ptr;
        for (ptr = raw + 2; *ptr; ptr++) {

```

```

        val *= 16;
        if (*ptr <= '9') {
            val += *ptr - '0';
        }
        else if (*ptr <= 'Z') {
            val += *ptr - 'A' + 10;
        }
        else {
            val += *ptr - 'a' + 10;
        }
    }
    buffer[BUFLen - 1] = 0;
    for (ptr = buffer + BUFLen - 2; val; ptr--) {
        *ptr = val % 10 + '0';
        val /= 10;
    }
    return ++ptr;
}
else {
    return raw;
}
}

```

4 Syntax Analysis

I used bison to generate a parser for BPL. It can generate an AST for a given BPL program by the given rules, and recover from syntax errors.

The implementation of syntax.y is as follows.

```

%{
    #include "lex.yy.c"
    #include "tree.h"
    #include "error.h"
    int exit_code = 0;
    Tree result = NULL;
    void yyerror(const char *s);
    void log_err(char type, size_t line_no, const char *msg);
%}

%union {
    struct Head *node;
}

%token <node> ID INT FLOAT CHAR STRUCT RETURN IF ELSE WHILE PLUS MINUS MUL DIV
AND OR LT LE GT GE NE EQ NOT ASSIGN TYPE LP RP LB RB LC RC SEMI COMMA DOT
ILLEGAL

```



```
%type <node> Program ExtDefList ExtDef ExtDeclList Specifier StructSpecifier
          VarDec FunDec VarList ParamDec CompSt StmtList Stmt DefList Def DeclList Dec
          Exp Args
%right ELSE
%left ASSIGN
%left AND OR
%left LT LE GT GE NE EQ
%left PLUS MINUS
%left MUL DIV
%right NOT
%left LP RP LB RB DOT
%%

/* high-level definition */
Program: ExtDefList {
    $$ = new_node("Program", $1->line_no, NULL, $1, NULL);
    result = $$;
}
ExtDefList: ExtDef ExtDefList {
    $1->sibling = $2;
    $$ = new_node("ExtDefList", $1->line_no, NULL, $1, NULL);
}
| /*empty*/ {
    $$ = new_node(NULL, 0, NULL, NULL, NULL);
}
ExtDef: Specifier ExtDeclList SEMI {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("ExtDef", $1->line_no, NULL, $1, NULL);
}
| Specifier SEMI {
    $1->sibling = $2;
    $$ = new_node("ExtDef", $1->line_no, NULL, $1, NULL);
}
| Specifier FunDec CompSt {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("ExtDef", $1->line_no, NULL, $1, NULL);
}
| Specifier ExtDeclList error {
    log_err(MISSING_SEMI, $2->line_no, "missing semicolon");
}
| Specifier error {
    log_err(MISSING_SEMI, $1->line_no, "missing semicolon");
}
```

```
    }
ExtDecList: VarDec COMMA ExtDecList {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("ExtDecList", $1->line_no, NULL, $1, NULL);
}
| VarDec {
    $$ = new_node("ExtDecList", $1->line_no, NULL, $1, NULL);
}

/* specifier */
Specifier: TYPE {
    $$ = new_node("Specifier", $1->line_no, NULL, $1, NULL);
}
| StructSpecifier {
    $$ = new_node("Specifier", $1->line_no, NULL, $1, NULL);
}

StructSpecifier: STRUCT ID LC DefList RC {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;
    $4->sibling = $5;
    $$ = new_node("StructSpecifier", $1->line_no, NULL, $1, NULL);
}
| STRUCT ID {
    $1->sibling = $2;
    $$ = new_node("StructSpecifier", $1->line_no, NULL, $1, NULL);
}

/* declarator */
VarDec: ID {
    $$ = new_node("VarDec", $1->line_no, NULL, $1, NULL);
}
| VarDec LB INT RB {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;
    $$ = new_node("VarDec", $1->line_no, NULL, $1, NULL);
}

FunDec: ID LP VarList RP {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;
    $$ = new_node("FunDec", $1->line_no, NULL, $1, NULL);
}
```

```
}
| ID LP RP {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("FunDec", $1->line_no, NULL, $1, NULL);
}
| ID LP VarList error {
    log_err(MISSING_CLOSING, $3->line_no, "missing closing symbol");
}
| ID LP error {
    log_err(MISSING_CLOSING, $2->line_no, "missing closing symbol");
}
VarList: ParamDec COMMA VarList {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("VarList", $1->line_no, NULL, $1, NULL);
}
| ParamDec {
    $$ = new_node("VarList", $1->line_no, NULL, $1, NULL);
}
ParamDec: Specifier VarDec {
    $1->sibling = $2;
    $$ = new_node("ParamDec", $1->line_no, NULL, $1, NULL);
}

/* statement */
CompSt: LC DefList StmtList RC {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;
    $$ = new_node("CompSt", $1->line_no, NULL, $1, NULL);
}
StmtList: Stmt StmtList {
    $1->sibling = $2;
    $$ = new_node("StmtList", $1->line_no, NULL, $1, NULL);
}
| /*empty*/ {
    $$ = new_node(NULL, 0, NULL, NULL, NULL);
}
| Stmt Def DefList StmtList error {
    log_err(DEF_AFTER_STMT, $2->line_no, "def after stmt");
}
Stmt: Exp SEMI {
    $1->sibling = $2;
```

```

    $$ = new_node("Stmt", $1->line_no, NULL, $1, NULL);
}
| CompSt {
    $$ = new_node("Stmt", $1->line_no, NULL, $1, NULL);
}
| RETURN Exp SEMI {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Stmt", $1->line_no, NULL, $1, NULL);
}
| RETURN Exp error {
    log_err(MISSING_SEMI, $2->line_no, "missing semicolon");
}
| IF LP Exp RP Stmt %prec ELSE {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;
    $4->sibling = $5;
    $$ = new_node("Stmt", $1->line_no, NULL, $1, NULL);
}
| IF LP Exp RP Stmt ELSE Stmt {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;
    $4->sibling = $5;
    $5->sibling = $6;
    $6->sibling = $7;
    $$ = new_node("Stmt", $1->line_no, NULL, $1, NULL);
}
| WHILE LP Exp RP Stmt {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;
    $4->sibling = $5;
    $$ = new_node("Stmt", $1->line_no, NULL, $1, NULL);
}
| Exp error {
    log_err(MISSING_SEMI, $1->line_no, "missing semicolon");
}
}

/* local definition */
DefList: Def DefList {
    $1->sibling = $2;
    $$ = new_node("DefList", $1->line_no, NULL, $1, NULL);
}

```

```
}  
| /*empty*/ {  
    $$ = new_node(NULL, 0, NULL, NULL, NULL);  
}  
Def: Specifier Declist SEMI {  
    $1->sibling = $2;  
    $2->sibling = $3;  
    $$ = new_node("Def", $1->line_no, NULL, $1, NULL);  
}  
| Specifier Declist error {  
    log_err(MISSING_SEMI, $2->line_no, "missing semicolon");  
}  
Declist: Dec {  
    $$ = new_node("Declist", $1->line_no, NULL, $1, NULL);  
}  
| Dec COMMA Declist {  
    $1->sibling = $2;  
    $2->sibling = $3;  
    $$ = new_node("Declist", $1->line_no, NULL, $1, NULL);  
}  
| ILLEGAL error {  
    log_err(LEXEME_ERROR, $1->line_no, $1->property);  
}  
Dec: VarDec {  
    $$ = new_node("Dec", $1->line_no, NULL, $1, NULL);  
}  
| VarDec ASSIGN Exp {  
    $1->sibling = $2;  
    $2->sibling = $3;  
    $$ = new_node("Dec", $1->line_no, NULL, $1, NULL);  
}  
  
/* Expression */  
Exp: Exp ASSIGN Exp {  
    $1->sibling = $2;  
    $2->sibling = $3;  
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);  
}  
| Exp AND Exp {  
    $1->sibling = $2;  
    $2->sibling = $3;  
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);  
}  
| Exp OR Exp {
```

```
$1->sibling = $2;
$2->sibling = $3;
$$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp LT Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp LE Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp GT Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp GE Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp NE Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp EQ Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp PLUS Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp MINUS Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
```

```

| Exp MUL Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp DIV Exp {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| LP Exp RP {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| MINUS Exp {
    $1->sibling = $2;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| NOT Exp {
    $1->sibling = $2;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| ID LP Args RP {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| ID LP RP {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| ID LP Args error {
    log_err(MISSING_CLOSING, $3->line_no, "missing closing symbol");
}
| ID LP error {
    log_err(MISSING_CLOSING, $2->line_no, "missing closing symbol");
}
| Exp LB Exp RB {
    $1->sibling = $2;
    $2->sibling = $3;
    $3->sibling = $4;

```

```
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| Exp DOT ID {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| ID {
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| INT {
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| FLOAT {
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| CHAR {
    $$ = new_node("Exp", $1->line_no, NULL, $1, NULL);
}
| ID ILLEGAL Exp error {
    log_err(ILLEGAL_OP, $2->line_no, "illegal operator");
}
| ILLEGAL error {
    log_err(LEXEME_ERROR, $1->line_no, $1->property);
}
}
Args: Exp COMMA Args {
    $1->sibling = $2;
    $2->sibling = $3;
    $$ = new_node("Args", $1->line_no, NULL, $1, NULL);
}
| Exp {
    $$ = new_node("Args", $1->line_no, NULL, $1, NULL);
}
%%

void yyerror(const char *s) { /* fprintf(stderr, "%s\n", s); */ }

void log_err(char type, size_t line_no, const char *msg) {
    fprintf(stdout, "Error type %c at Line %lu: %s\n", type, line_no, msg);
    exit_code = 1;
}

int main(int argc, char **argv) {
    if (argc != 2) {
```



```
    exit(1);
}
FILE *fin = fopen(argv[1], "rb"), *fout = stdout;
if (!fin) {
    exit(1);
}
size_t len = strlen(argv[1]);
if (!strcmp(argv[1] + len - 4, ".bp1")) {
    strcpy(argv[1] + len - 3, "out");
    fout = freopen(argv[1], "wb", stdout);
}
if (!fout) {
    exit(1);
}
yyin = fin;
yyparse();
if (!exit_code) {
    print_tree(stdout, result, 0);
}
empty_tree(result);
if (result) {
    free(result);
    result = NULL;
}
return exit_code;
}
```

4 Error Handling

Two constants, LEXEME_ERROR and SYNTAX_ERROR, are defined to represent errors with the characters 'A' and 'B', respectively. LEXEME_ERROR indicates errors encountered during lexical analysis, such as illegal characters or illegal hexadecimal numbers. SYNTAX_ERROR indicates errors encountered during syntax analysis, such as missing semicolons or closing braces.

ILLEGAL_CHARSET, ILLEGAL_HEX, ILLEGAL_CHAR, ILLEGAL_ID, and ILLEGAL_OP are all lexical errors, sharing the value of LEXEME_ERROR. MISSING_SEMI, MISSING_CLOSING, and DEF_AFTER_STMT are syntax errors, sharing the value of SYNTAX_ERROR.

The implementation of errlist.h is as follows.

```
#ifndef ERRLIST_H_
#define ERRLIST_H_

#define LEXEME_ERROR 'A'
```

```
#define SYNTAX_ERROR 'B'

enum Error {
    ILLEGAL_CHARSET = LEXEME_ERROR,
    ILLEGAL_HEX      = LEXEME_ERROR,
    ILLEGAL_CHAR     = LEXEME_ERROR,
    ILLEGAL_ID       = LEXEME_ERROR,
    ILLEGAL_OP       = LEXEME_ERROR,
    MISSING_SEMI     = SYNTAX_ERROR,
    MISSING_CLOSING  = SYNTAX_ERROR,
    DEF_AFTER_STMT   = SYNTAX_ERROR,
};

#endif
```

5 Optional Feature

Except for the required features, I also implemented the following optional feature. My parser can recognize both single-line and multi-line comments.

For the single-line comments, I simply ignored them. For the multi-line comments, I counted the number of newlines in the comment and added it to the line number.

```
"/".*$      { /* ignore comments */ }
"/*"([^\]|[*][^/]|[*][*][^/])*/" { for (size_t i = 0; i < strlen(yytext); i++) if
(yytext[i] == '\n') line++; }
```

6 Run

6.1 Make

The command was used:

```
make bplc
root@6eca9c2b53e9:/mnt/Workspace/project/project1# make bplc
flex lex.l
bison -t -d syntax.y
mkdir bin
gcc -o bin/bplc -lfl -ly syntax.tab.c tree.c
```

6.2 Test

6.2.1

Test test case one by one.

Test test/test_1_r01.bpl. The command was used:

```
./bin/bplc ./test/test_1_r01.bpl
root@6eca9c2b53e9:/mnt/Workspace/project/project1# ./bin/bplc ./test/test_1_r01.bpl
```

We can find that the output "test_1_r01.out" has been generated in the test folder.

 test_1_r01.bpl	2024/10/29 14:46	BPL 文件	1 KB
 test_1_r01.out	2024/10/30 17:34	OUT 文件	2 KB

We can test the other test cases to generate output files in the same way.

These commands were used:

```
./bin/bplc ./test/test_1_r02.bpl
./bin/bplc ./test/test_1_r03.bpl
./bin/bplc ./test/test_1_r04.bpl
./bin/bplc ./test/test_1_r05.bpl
./bin/bplc ./test/test_1_r06.bpl
./bin/bplc ./test/test_1_r07.bpl
./bin/bplc ./test/test_1_r08.bpl
./bin/bplc ./test/test_1_r09.bpl
./bin/bplc ./test/test_1_r10.bpl
./bin/bplc ./test/test_1_r11.bpl
./bin/bplc ./test/test_1_r12.bpl
```

6.2.2

Of course, we can test all test cases by running test.py.

The implementation of test.py is as follows.

```
import subprocess

for i in range(1,13):
    if i<=9:
        name = f"0{i}"
    else:
        name = f"{i}"
    file = f"test/test_1_r{name}.bpl"
    command = [ "./bin/bplc", file ]
    print(file+" OK.")
    subprocess.run(command)
```

The command was used:

```
python3 test.py
```

```
root@6eca9c2b53e9:/mnt/Workspace/project/project1# python3 test.py
test/test_1_r01.bpl OK.
test/test_1_r02.bpl OK.
test/test_1_r03.bpl OK.
test/test_1_r04.bpl OK.
test/test_1_r05.bpl OK.
test/test_1_r06.bpl OK.
test/test_1_r07.bpl OK.
test/test_1_r08.bpl OK.
test/test_1_r09.bpl OK.
test/test_1_r10.bpl OK.
test/test_1_r11.bpl OK.
test/test_1_r12.bpl OK.
root@6eca9c2b53e9:/mnt/Workspace/project/project1#
```

We can find that the all output files have been generated in the test folder.

test_1_r01.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r01.out	2024/10/30 17:42	OUT 文件	2 KB
test_1_r02.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r02.out	2024/10/30 17:42	OUT 文件	1 KB
test_1_r03.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r03.out	2024/10/30 17:42	OUT 文件	3 KB
test_1_r04.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r04.out	2024/10/30 17:42	OUT 文件	1 KB
test_1_r05.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r05.out	2024/10/30 17:42	OUT 文件	2 KB
test_1_r06.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r06.out	2024/10/30 17:42	OUT 文件	1 KB
test_1_r07.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r07.out	2024/10/30 17:42	OUT 文件	1 KB
test_1_r08.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r08.out	2024/10/30 17:42	OUT 文件	1 KB
test_1_r09.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r09.out	2024/10/30 17:42	OUT 文件	3 KB
test_1_r10.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r10.out	2024/10/30 17:42	OUT 文件	1 KB
test_1_r11.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r11.out	2024/10/30 17:42	OUT 文件	1 KB
test_1_r12.bpl	2024/10/29 14:46	BPL 文件	1 KB
test_1_r12.out	2024/10/30 17:42	OUT 文件	2 KB

7 Results

7.1 Syntactically Valid BPL Program

For a syntactically valid BPL program, my parser can print out its syntax tree.

e.g. test_1_r03.bpl

```
int global;
struct my_struct
{
    int code;
    char data;
};
int test_1_r03()
{
    struct my_struct obj;
    obj.code = global;
    global = global + 1;
}
```

My parser can print its corresponding syntax tree as follow. (test_1_r03.out)

```
Program (1)
  ExtDefList (1)
    ExtDef (1)
      Specifier (1)
        TYPE: int
      ExtDecList (1)
        VarDec (1)
          ID: global
      SEMI
    ExtDefList (2)
      ExtDef (2)
        Specifier (2)
          StructSpecifier (2)
            STRUCT
            ID: my_struct
            LC
          DefList (4)
            Def (4)
              Specifier (4)
                TYPE: int
              Declist (4)
                Dec (4)
                  VarDec (4)
                    ID: code
              SEMI
            DefList (5)
              Def (5)
                Specifier (5)
                  TYPE: char
                Declist (5)
```

```
        Dec (5)
        VarDec (5)
        ID: data
    SEMI
RC
SEMI
ExtDefList (7)
ExtDef (7)
    Specifier (7)
    TYPE: int
FunDec (7)
    ID: test_1_r03
    LP
    RP
CompSt (8)
    LC
    DefList (9)
    Def (9)
    Specifier (9)
    StructSpecifier (9)
    STRUCT
    ID: my_struct
    DeclList (9)
    Dec (9)
    VarDec (9)
    ID: obj
    SEMI
    StmtList (10)
    Stmt (10)
    Exp (10)
    Exp (10)
    Exp (10)
    ID: obj
    DOT
    ID: code
    ASSIGN
    Exp (10)
    ID: global
    SEMI
    StmtList (11)
    Stmt (11)
    Exp (11)
    Exp (11)
    ID: global
```

```

    ASSIGN
    Exp (11)
    Exp (11)
    ID: global
    PLUS
    Exp (11)
    INT: 1
    SEMI
    RC
    
```

1	Program (1)	29	TYPE: char	57	SEMI
2	ExtDefList (1)	30	Declist (5)	58	StmtList (10)
3	ExtDef (1)	31	Dec (5)	59	Stmt (10)
4	Specifier (1)	32	VarDec (5)	60	Exp (10)
5	TYPE: int	33	ID: data	61	Exp (10)
6	ExtDeclist (1)	34	SEMI	62	Exp (10)
7	VarDec (1)	35	RC	63	ID: obj
8	ID: global	36	SEMI	64	DOT
9	SEMI	37	ExtDefList (7)	65	ID: code
10	ExtDefList (2)	38	ExtDef (7)	66	ASSIGN
11	ExtDef (2)	39	Specifier (7)	67	Exp (10)
12	Specifier (2)	40	TYPE: int	68	ID: global
13	StructSpecifier (2)	41	FunDec (7)	69	SEMI
14	STRUCT	42	ID: test_1_r03	70	StmtList (11)
15	ID: my_struct	43	LP	71	Stmt (11)
16	LC	44	RP	72	Exp (11)
17	DefList (4)	45	CompSt (8)	73	Exp (11)
18	Def (4)	46	LC	74	ID: global
19	Specifier (4)	47	DefList (9)	75	ASSIGN
20	TYPE: int	48	Def (9)	76	Exp (11)
21	Declist (4)	49	Specifier (9)	77	Exp (11)
22	Dec (4)	50	StructSpecifier (9)	78	ID: global
23	VarDec (4)	51	STRUCT	79	PLUS
24	ID: code	52	ID: my_struct	80	Exp (11)
25	SEMI	53	Declist (9)	81	INT: 1
26	DefList (5)	54	Dec (9)	82	SEMI
27	Def (5)	55	VarDec (9)	83	RC
28	Specifier (5)	56	ID: obj	84	

7.2 Syntactically Invalid BPL Program

For a syntactically invalid BPL program, my parser can report all errors.

e.g. test_1_r10.bpl

```

int test_1_r10()
{
    int a, b = 1;
    a = 10;
    int i = 0;
    while (i < a)
    {
        b = b + i;
        i = i + 1;
    }
}
    
```

My parser can report all errors: one syntax error (Error type B). (test_1_r10.out)

Error type B at Line 5: def after stmt

```

1  Error type B at Line 5: def after stmt
2
    
```

e.g. test_1_r07.bpl

```
int test_1_r07()  
{  
    int a, b, c;  
    a = 1;  
    b = a && 2;  
    c = b || 3;  
    a = c | 4;  
    b = a & 5;  
}
```

My parser can report all errors: two lexical error (Error type A). (test_1_r07.out)

```
Error type A at Line 7: illegal operator  
Error type A at Line 8: illegal operator
```

```
1   Error type A at Line 7: illegal operator  
2   Error type A at Line 8: illegal operator  
3
```